EDGE-AI ENABLED IOT ARCHITECTURE FOR EARLY FIRE DETECTION IN SMART AGRICULTURE

A project report submitted in partial fulfilment of requirement for the award of degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

SUBMITTED BY

G. TANUJA SAI SWATHI	**(22B81A0452)**
B. SRI RAJENDRA	**(22B81A0422)**
CH. RAJ KUMAR	**(22B81A0426)**
D. AJAY	**(22B81A0441)**
B. RAMESH	**(22B81A0420)**

Under the guidance of

Mr. H. Sudhakar M. Tech, (Ph. D)

Assistant Professor, Department of ECE

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

SIR C.R. REDDY COLLEGE OF ENGINEERING,

ELURU-534 007, ELURU DISTRICT, A.P.

(Affiliated to JNTU Kakinada, Accredited by NBA & NAAC)

(2025-2026)

SIR C.R. REDDY COLLEGE OF ENGINEERING

ELURU-534 007, ELURU DISTRICT., A.P.

(Affiliated to **JNTU Kakinada**, Accredited by NAAC & NBA)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that the project work title "**EDGE-AI ENABLED IOT ARCHITECTURE FOR EARLY FIRE DETECTION IN SMART AGRICULTURE"** has been successfully completed by the following students.

G. TANUJA SAI SWATHI

(22B81A0452)

B. SRI RAJENDRA

(22B81A0422)

CH. RAJKUMAR

(22B81A0426)

D. AJAY

(22B81A0441)

B. RAMESH

(22B81A0420)

During the academic year 2024-2025 as a partial fulfilment of the requirement for the award of **BACHELOR OF TECHNOLOGY** degree in **ELECTRONICS AND COMMUNICATION ENGINEERING** of JNTU Kakinada**.**

PROJECT GUIDE

Mr. H. SUDHAKAR, M. Tech, (Ph. D)

Assistant professor,ECE

HEAD OF THE DEPARTMENT

Dr. P.H.S TEJO MURTHI, M. E, Ph. D.

Professor & HOD-ECE

EXTERNAL EXAMINE

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any work would be incomplete without mentioning the people who made it possible and whose encouragement and guidance has been a source of inspiration throughout the course of the project.

We express our sincere thanks to **Mr. H. SUDHAKAR**, M. Tech, (Ph. D), Assistant professor, Dept. of ECE for being a source of inspiration and for constantly encouraging us throughout the course to pursue new goals and ideas to implement this project.

We are immensely thankful to our **Dr. M. RAVI KUMAR**, Professor, and **Dr. I. HEMALATHA** Associate Professor, Department of Electronics and Communication Engineering for giving us valuable suggestions and their co-ordinance.

We are privileged to express our sincere gratitude to honorable Head of the Electronics & Communication Engineering Department, **Dr. P. H S TEJO MURTHY** for giving his continuous support and guidance in our endeavors.

We take the opportunity to express our heartfelt gratitude to our beloved principal **Dr. K. VENKATESWARA RAO** for giving us an opportunity to fulfil our aspirations.

Our sincere thanks to **Smt. D. LAKSHMI KUMARI**, Central Librarian and other staff of our college for providing the necessary resources.

PROJECT MEMBERS

G. TANUJA SAI SWATHI	(22B81A0452)
B. SRI RAJENDRA	(22B81A0422)
CH. RAJKUMAR	(22B81A0426)
D.AJAY	(22B81A0441)
B. RAMESH	(22B81A0420)

INDEX

CONTENTS PAGE NO

Abstract i

List of Tables ii

List of Figures iii

Chapter 1: Introduction 1

1.1 Background and Motivation 2

1.2 Problem Statement 2

1.3 Objectives 3

1.4 Need for the System 4

1.5 Scope of the Project 6

Chapter 2: Literature Survey 7

2.1 Traditional Fire Detection Systems 8

2.2 IoT in Agriculture 8

2.3 AI-based Fire Detection 9

2.4 Edge Computing 10

2.5 Limitations of existing 10

2.6 Comparative Analysis and Research Gaps 11

2.7 Authors analysis 12

Chapter 3: Proposed System 14

3.1 System Overview 15

3.2 System Architecture 15

3.3 Three – layer IoT Architecture Justification 16

3.4 Working Principle 17

3.5 Block Diagram 20

3.6 Advantages of Proposed System 20

3.7 Differences between Referred system & Proposed system 21

Chapter 4: Hardware Implementation 22

4.1 Raspberry Pi 4 Model B 23

4.2 DHT11 Temperature & Humidity Sensor 24

4.3 Flame Sensor (IR-based) 25

4.4 USB Camera / Pi Camera Module 26

4.5 Power Supply 26

4.6 Circuit Connections 27

Chapter 5: Software Stack 28

5.1 Python 3 29

5.2 Flask Web Framework 30

5.3 OpenCV 30

5.4 YOLOv8 Model 30

5.5 Frontend Technologies 31

5.6 Prototype 32

Chapter 6: Software Implementation 33

6.1 Sensor Integration Code 34

6.2 Fire Detection Algorithm 34

6.3 Web Dashboard Implementation 35

6.4 Remote desktop tool 35

6.5 Multithreading Architecture 35

6.6 Fire detection Alert 36

Chapter 7: Edge Processing & IoT 37

7.1 Edge Computing Paradigm 38

7.2 Edge VS Cloud computing comparison 38

7.3 IoT Communication 38

7.4 Alert System 38

Chapter 8: Results & Analysis 41

8.1 Testing Methodology 42

8.2 Performance Observations 42

8.3 Graph Analysis 43

8.4 Data Processing & Visualization Approach 45

8.5 Output 45

8.6 Calculations 46

Chapter 9: Conclusion & Future Scope 48

9.1 Conclusion 49

9.2 Future Scope 49

References 52

Appendix A: Components List 53

Appendix B: Source Code Snippets 54

Appendix C: Key Code Snippets 55

ABSTRACT

Architecture for Early Fire Detection in Smart Agriculture

Agricultural fields are highly vulnerable to fire hazards due to extreme temperatures, drought conditions, dry vegetation, lightning, equipment faults, and human activities. Rapid fire spread in open environments makes early detection essential to prevent severe crop damage and economic loss.

This project presents an Edge-AI enabled IoT-based fire detection system designed for smart agriculture. It integrates multiple sensors, including DHT11 for temperature and humidity, flame sensors for infrared detection, and a camera module for visual monitoring, creating a multi-layered detection approach.

A deep learning model based on the YOLO architecture is deployed on a Raspberry Pi 4 to perform real-time fire detection from video streams. The model identifies fire regions with bounding boxes and generates near-instant alerts.

The system uses edge computing, processing all data locally on the Raspberry Pi, which reduces latency, minimizes bandwidth usage, and removes dependency on internet connectivity. A Flask-based web dashboard provides live video, real-time sensor data, and alert notifications accessible over a local network.

Testing under controlled conditions shows reliable early fire detection with low latency and effective alert generation. The system offers a cost-effective, scalable solution for improving fire safety in agricultural environments.

Keywords: Edge Computing, IoT, Fire Detection, YOLO, Deep Learning, Raspberry Pi, Smart Agriculture, Flask, OpenCV, DHT11, Real-time Monitoring.

LIST OF TABLES

Table Name	Page No
2.6 Comparative Analysis and Research Gaps	11
2.7 Authors Analysis	12
3.6 Advantages of Proposed System	20
4.2 DHT11 Temperature & Humidity Sensor	24
4.3 Flame Sensor Module (IR-based)	25
4.6 Circuit Connections	27
6.5 Multithreading Architecture	35
7.4.3 Alert Logging	40
8.2 Performance Observations	42
APPENDIX A	53
APPENDIX B	54
APPENDIX C	55

LIST OF FIGURES

Figure Name	Page No
3.1 System Overview	15
3.4 Flow Chart of User Notification	19
3.5 Block Diagram	20
4.1. Raspberry Pi 4 Model B	23
4.2. DHT11 Temperature & Humidity Sensor	24
4.3 Flame Sensor Module	25
5.1. Python Code	29
5.4 YOLOv8 Model (Ultralytics)	31
5.6 Prototype	32
6.6 Fire detection Alert	36
7.4.1 Visual Alerts	39
8.3.1,8.3.2 Temperature and Humidity VS Time	44
8.3.3 YOLO Confidence VS Time	44

CHAPTER 1

INTRODUCTION

1.1Back ground:

Agriculture remains a critical sector in both developing and developed economies. In India, it contributes around 17–18% to GDP and employs over 50% of the population. With rising food demand, protecting agricultural assets such as crops, stored grain, and equipment from hazards like fire has become essential.

Fires in agricultural fields can arise from various sources, including dry stubble combustion, electrical faults, lightning, and intentional burning. Due to open-field conditions and wind, fires spread rapidly, causing severe economic losses and disrupting food supply chains. Traditional monitoring methods are reactive and often detect fires only after significant damage has occurred.

To address this gap, automated fire detection systems are needed. Advances in IoT, deep learning, and edge computing enable the development of intelligent, real-time monitoring solutions. This project proposes an Edge-AI based IoT system for early fire detection in agriculture.

Unlike cloud-dependent systems, this approach processes data locally at the edge, ensuring faster response and improved reliability. By combining computer vision with environmental sensing, the system reduces false alarms and enables continuous, low-cost monitoring suitable for large-scale deployment. It represents a step toward autonomous and intelligent agricultural safety infrastructure.

1.2 Problem Statement

Agricultural fires are a persistent and economically devastating hazard. Existing detection approaches suffer from several critical shortcomings:

1. Delayed Detection: Manual monitoring is reactive and unable to detect fire at ignition. By the time smoke or flames are visible at a distance, the fire has already grown substantially.

2. False Alarms in Open Environments: Simple heat or smoke sensors designed for indoor environments produce excessive false alarms in agricultural fields due to ambient heat, dust, and natural environmental variations.
3. Lack of Intelligent Analysis: Conventional sensor-only systems provide no contextual or visual confirmation of fire events, leading to alert fatigue and slow response.
4. Cloud Dependency and Latency: Systems that rely on cloud processing for AI-based detection introduce unacceptable network latency and are vulnerable to connectivity failures common in rural agricultural areas.
5. Scalability Challenges: Existing commercial solutions are expensive and difficult to scale across large agricultural holdings.

There is a clear need for an integrated, intelligent, locally-processing fire detection system that provides early warning, multi-sensor confirmation, and remote monitoring in agricultural environments

1.3 Objectives

The primary objectives of this project are:

1. To develop a real-time fire detection system leveraging a YOLO deep learning model deployed on edge hardware (Raspberry Pi 4).
2. To integrate multiple environmental sensors (DHT11 for temperature/humidity, flame sensor for IR radiation) for multi-modal fire confirmation.
3. To implement edge computing architecture ensuring low-latency, local data processing without dependency on cloud services.
4. To develop a web-based monitoring dashboard using Flask providing live video, sensor graphs, and fire alert notifications.
5. To evaluate the system's detection performance and response time in controlled test conditions.

1.4 Need for the System

The proposed fire detection system is designed to address critical gaps in agricultural safety, where traditional monitoring methods are insufficient to prevent large-scale losses and hazards. Agricultural environments are highly vulnerable to fire incidents due to dry vegetation, extreme temperatures, and lack of continuous supervision. The need for an automated, intelligent monitoring system arises from the following key factors:

Crop Loss Prevention: Agricultural fires result in significant economic losses each year, particularly during dry seasons when crops become highly flammable.

1. A single fire incident can destroy **entire harvest cycles within minutes**, leading to irreversible financial damage.
2. Farmers often invest months of labor and resources into cultivation, making fire incidents devastating.
3. Early detection systems can:
    - Identify fire at its initial stage
    - Trigger alerts before the fire spreads uncontrollably
    - Enable quicker response and containment

This system minimizes damage by reducing detection time from minutes (human response) to seconds (automated detection).

Farmer Safety: Human safety is a critical concern in agricultural environments. Farm workers operating in fields may not notice fire outbreaks immediately due to:

1. Large field areas
2. Obstructed visibility
3. Environmental distractions
4. Delayed awareness increases the risk of:
    - Burns and injuries
    - Fatalities in extreme cases

The proposed system provides **instant alerts**, allowing:

- Early evacuation
- Safer working conditions
- Reduced dependency on manual vigilance

Remote Monitoring: Modern agricultural operations often span large and geographically dispersed areas.

1. Farmers and landowners cannot monitor all sections of land continuously.
2. Traditional methods rely on periodic inspection, which is unreliable for sudden fire events.
3. By integrating a **Flask-based web dashboard**, the system enables:
4. Real-time monitoring from smartphones or computers
5. Live video feed of field conditions
6. Instant access to alerts regardless of location

This transforms fire monitoring from a local task into a remotely manageable system.

Cost-Effectiveness: Existing fire detection systems are often expensive and not tailored for agricultural use.

- Industrial fire detection systems:
  - Require high installation and maintenance costs
  - Are not feasible for small-scale farmers

The proposed system uses:

1. Low-cost sensors (DHT11, flame sensor)
2. Raspberry Pi as an affordable computing unit
3. Open-source software frameworks

This ensures:

1. Accessibility for small and medium-scale farmers
2. Scalability to larger farms by deploying multiple units

24/7 Operation: Human monitoring is inherently limited by fatigue, attention span, and availability.

- Fires can occur at any time, including:
  - Late night hours
  - Early morning periods when supervision is minimal

The automated system provides:

1. Continuous monitoring without interruption
2. Consistent performance regardless of time or conditions
3. Immediate detection and response

This eliminates the dependency on human presence for safety.

1.5 Scope of the Project

This project covers the following aspects:

1. Hardware setup and integration of Raspberry Pi 4, DHT11, flame sensor, and camera module.
2. Training/fine-tuning and deployment of YOLOv8 fire detection model on edge hardware.
3. Development of Flask-based web server and real-time monitoring dashboard.
4. Implementation of multithreaded Python application for concurrent sensor reading, video processing, and web serving.
5. Testing and performance evaluation under simulated fire conditions.

The project does not cover: large-scale multi-node distributed deployments, GSM/cellular alert integration (identified as future work), or outdoor weatherproofing enclosure design, though these are acknowledged as natural extensions of the current work

CHAPTER 2

LITERATURE SURVEY

This chapter reviews existing research and systems related to fire detection in agricultural and open environments, IoT applications in smart farming, AI-based fire detection using computer vision, and the role of edge computing in real-time IoT systems.

2.1 Traditional Fire Detection Systems

Conventional fire detection systems, developed in the early 20th century, rely on sensing combustion by-products such as smoke, heat, or carbon monoxide using fixed sensors. Photoelectric detectors identify smoke through light scattering, ionization detectors detect changes in ionized air, while heat detectors respond to fixed temperature thresholds or rapid temperature increases.

Although effective in enclosed environments like buildings and warehouses, these systems are unsuitable for open agricultural fields. In such settings, smoke and heat dissipate quickly due to large open spaces and wind, preventing sensors from reaching detection thresholds. Additionally, high ambient temperatures can cause false alarms in heat detectors, while dust and pollen interfere with smoke detection.

Studies have shown that conventional point-sensor systems result in delayed detection and high false alarm rates in outdoor environments, making them inadequate for agricultural fire monitoring.

2.2 IoT in Agriculture

The Internet of Things has seen growing application in precision agriculture over the past decade. IoT-based agricultural systems deploy distributed sensor networks across fields to monitor soil moisture levels, ambient and soil temperature, humidity, rainfall, wind speed, and crop health indicators. These systems transmit data wirelessly to central servers or cloud platforms for analysis and visualization.

Projects such as smart irrigation systems triggered by soil moisture IoT sensors have demonstrated 30-40% reductions in water usage while maintaining crop yields. Weather monitoring IoT systems provide hyper-local meteorological data to inform

farming decisions. Livestock tracking systems using IoT tags improve herd management efficiency.

However, existing IoT agricultural deployments focus primarily on crop productivity and resource optimization, not safety. Fire detection has received relatively little attention in IoT agricultural research, representing a significant gap that this project addresses. Furthermore, most IoT agricultural systems rely on cloud connectivity for data processing and storage, which is problematic in rural areas with unreliable internet access.

2.3 AI-based Fire Detection

The application of computer vision and deep learning to fire detection has been an active area of research since the 2000s. Early approaches used color-based segmentation algorithms to identify pixel regions matching the color profile of fire (reds, oranges, yellows) in video frames. While computationally efficient, color-based methods produce significant false positives from objects such as sunsets, street lighting, and orange-colored objects.

Subsequent research introduced motion analysis and texture-based features combined with color features to improve accuracy. The advent of Convolutional Neural Networks (CNNs) in the 2010s enabled significantly more robust fire classification from image patches. Researchers proposed architectures for fire region segmentation using CNNs with custom fire datasets.

The introduction of real-time object detection models, particularly the YOLO (You Only Look Once) family of models beginning with YOLOv1 in 2016, enabled fire detection as an object localization problem — identifying not just whether fire is present but precisely where it appears in the frame. YOLO models process entire images in a single forward pass through the neural network, enabling detection at video frame rates on GPU hardware.

YOLOv8, released by Ultralytics, represents the current state-of-the-art in the YOLO family, offering improved accuracy, smaller model size options suitable for edge deployment, and comprehensive Python API support. Several researchers have reported

successful fire detection with YOLOv8 models fine-tuned on fire datasets achieving mAP (mean Average Precision) scores above 0.85.

2.4 Edge Computing

Edge computing refers to the paradigm of processing data at or near the source of data generation — on embedded devices, single-board computers, or edge servers — rather than transmitting raw data to centralized cloud infrastructure for processing. This approach offers critical advantages for real-time safety systems:

1. Latency Reduction: Processing locally eliminates network round-trip time, reducing alert generation time from seconds to milliseconds.
2. Bandwidth Conservation: Only processed results (alerts, aggregated data) need to be transmitted, rather than raw video streams.
3. Reliability: Local processing continues to function even when network connectivity is lost, which is critical for safety systems.
4. Privacy: Sensitive video data remains on-premises rather than being transmitted to third-party cloud servers.

Edge computing is implemented in our system through the Raspberry Pi, which performs real-time fire detection using YOLOv8 locally without sending data to the cloud, ensuring low latency and offline operation.

The edge computing workflow begins with real-time data acquisition from sensors and camera modules. This data is processed locally on the Raspberry Pi using computer vision and machine learning algorithms. The YOLOv8 model performs fire detection directly on the device, and decisions are made instantly without relying on cloud servers. Alerts are generated locally and transmitted to users via a web interface over a local network.

2.5 Limitations of Existing System

Despite significant advancements in fire detection technologies, existing systems exhibit several limitations, particularly in agricultural environments.

Traditional fire detection systems rely primarily on smoke and heat sensors, which are ineffective in open-field conditions due to rapid dissipation of smoke and fluctuating environmental temperatures. These systems also suffer from high false alarm rates caused by dust, sunlight, and ambient heat variations.

IoT-based fire detection systems improve monitoring capabilities but often depend heavily on cloud infrastructure for data processing. This introduces latency and makes the system vulnerable to network failures, which are common in rural agricultural areas.

AI-based fire detection approaches using computer vision offer improved accuracy; however, many implementations rely on cloud-based processing or require high computational resources, limiting their deployment in low-cost embedded systems.

Furthermore, most existing solutions lack **multi-modal sensor fusion**, where environmental sensors and vision-based detection are combined to improve reliability. They also fail to implement **edge computing architectures**, resulting in slower response times and reduced system robustness.

These limitations highlight the need for a system that integrates sensor data, real-time AI-based detection, and edge computing to provide accurate, low-latency, and reliable fire detection in agricultural environments.

2.6 Comparative Analysis and Research Gaps

Approach	Advantages	Limitations
Traditional Sensors	Simple, low cost	High false alarm rate in open fields
Cloud-based AI	Powerful processing	Latency, connectivity dependency

UAV/Drone monitoring	Wide area coverage	High cost, limited flight time
Satellite remote sensing	Macro-scale detection	Low temporal resolution (hours)
Proposed Edge-AI IoT	Low latency, offline, intelligent	Limited single-point coverage

Despite the availability of various fire detection approaches, no single solution effectively addresses the combined challenges of accuracy, latency, reliability, and cost in agricultural environments. Traditional sensor-based systems suffer from high false alarm rates due to environmental variability, while cloud-based AI solutions introduce latency and depend heavily on stable internet connectivity. Furthermore, most existing systems lack integration between environmental sensing and intelligent visual analysis, resulting in limited contextual understanding of fire events. This creates a significant research gap for a system that combines multi-sensor data fusion, real-time AI-based detection, and edge computing to deliver fast, accurate, and reliable fire detection. The proposed system addresses this gap by integrating low-cost hardware, local processing, and intelligent algorithms into a unified, scalable architecture suitable for real-world agricultural deployment

.

2.7 Authors Analysis

Author	Year	Methodology	Advantages	Limitations
Ramadan et al.	**2024**	**UAV-based early wildfire detection using thermal + RGB cameras, onboard SBCs, and lightweight CNNs at the edge**	**Fast detection over large areas; edge AI reduces latency;**	**Limited drone flight time and energy; less ground-level sensor fusion**

			scalable coverage	
Morchid et al.	2024	IoT-enabled fire detection in sustainable agriculture using , multi-sensors, ThingSpeak/cloud, classical ML & embedded DL	Real-time detection; dashboards for farmers; sustainable design	Prototype scale; needs stronger mesh resilience and sensor fusion
Titu et al.	2024	Real-time fire detection using drones with edge computing and pruned CNN/YOLO models	Edge inference reduces latency and backhaul; good accuracy	Focuses on visual detection only; lacks smoke/gas sensor fusion
Aliyu et al.	2023	Blockchain-based trusted logging of IoT sensor data over LoRa/Cellular gateways	Tamper-proof audit trail; improved trustworthiness	Scalability & transaction cost issues for constrained nodes

CHAPTER 3

PROPOSED SYSTEM

3.1 System Overview

The proposed system is a comprehensive, integrated fire detection and monitoring platform designed for deployment in agricultural environments. At its core, the system employs a Raspberry Pi 4 as the central processing unit, which orchestrates data collection from multiple sensors, runs the AI fire detection model, serves the web dashboard, and manages alerts.

System Components at a Glance
Sensing Layer: DHT11 (Temperature & Humidity) + Flame Sensor (IR Radiation)
Vision Layer: USB Camera or Raspberry Pi Camera Module V2
Processing Layer: Raspberry Pi 4 (4GB RAM) running Python 3 AI Layer: YOLOv8 nano/small model for real-time fire detection Application Layer: Flask web server with real-time dashboard Presentation Layer: HTML5/CSS3/JavaScript frontend with live graphs

The system operates continuously in a multi-threaded mode: one thread captures video frames and runs YOLO inference, a second thread reads environmental sensor data, and a third thread serves the web dashboard to connected clients. This parallelism ensures that fire detection, sensor monitoring, and user interface responsiveness occur concurrently without blocking each other.

3.2 System Architecture

The architecture follows a layered design pattern:

1. Physical/Sensing Layer: Hardware sensors physically connected to the Raspberry Pi GPIO pins collect real-world environmental data (temperature,

humidity, flame radiation intensity).

2. Perception Layer: The camera captures visual information at configurable frame rates. OpenCV handles frame acquisition and preprocessing (resizing, color space conversion) before feeding frames to the YOLO model.
3. Intelligence Layer: The YOLOv8 model performs inference on each frame, returning bounding box coordinates, class labels, and confidence scores for detected fire regions. Detections above a configurable confidence threshold trigger fire alerts.
4. Integration Layer: A Python application thread integrates sensor data and vision detections, applying fusion logic — for example, treating flame sensor + YOLO detection as high-confidence fire vs. YOLO alone as medium-confidence.
5. Service Layer: The Flask web server exposes RESTful API endpoints and a streaming video endpoint over the local network, making data accessible to dashboard clients.
6. Presentation Layer: The HTML/JavaScript dashboard polls the Flask API periodically, displays live video via MJPEG stream, renders time-series charts of temperature and humidity using Chart.js, and presents fire alert panels with timestamp and confidence information.

3.3 Three-Layer IoT Architecture Justification

The proposed system follows a layered architecture inspired by modern IoT frameworks, consisting of the device layer, edge processing layer, and application layer.

1. **Device Layer:**
   Includes sensors such as DHT11 and flame sensor along with the camera module, responsible for real-time environmental data acquisition.
2. **Edge Processing Layer:**
   The Raspberry Pi performs local data processing, YOLO-based fire detection, and

sensor fusion. This reduces dependency on cloud infrastructure and ensures low-latency response.

3. **Application Layer:**
   The Flask web dashboard enables visualization of sensor data, video feed, and alert status for remote monitoring.

This layered design improves system modularity, scalability, and reliability, aligning with modern IoT architectures discussed in recent research.

3.4 Working Principle

The system operates through the following sequence of operations:

1. **System Initialization**
   At startup, the system initializes all hardware and software components to ensure smooth operation. The Raspberry Pi GPIO pins are configured, and sensors such as DHT11 and the flame sensor are set up. The camera module is activated using OpenCV, and the YOLOv8 model is loaded for real-time detection. A Flask web server is launched for monitoring, and multi-threading is enabled to handle sensing, processing, and alerts concurrently. This ensures the system is fully prepared for continuous operation.

2. Continuous Sensor Polling: A dedicated sensor thread continuously collects environmental data with controlled timing intervals.

1. Sensors (DHT11 and flame) are continuously monitored.
2. Data is stored safely and validated to remove noise.
3. Provides real-time environmental awareness.
4. **Frame Capture and Inference:** The vision processing thread continuously captures and analyzes video frames in real time.
5. Frames are captured (15–30 FPS) and preprocessed using OpenCV.
6. Processed frames are analyzed by YOLOv8 for fire and smoke detection.
7. Outputs include bounding boxes, class labels, and confidence scores.
8. Inference runs in ~100–300 ms on Raspberry Pi, enabling real-time detection.

3.Alert Generation: The system does not rely on a single input source. Instead, it uses a fusion-based decision mechanism.

1. Alert is triggered if YOLOv8 detects fire (above threshold) or the flame sensor is activated.
2. Validation includes temporal consistency and sensor cross-verification.
3. On detection, the system logs the event, updates alert status, and activates a cooldown period to avoid false toggling.
4. This ensures reliable detection with reduced false positives.

.**4.Dashboard Update:** Once a fire condition is confirmed, the system initiates response mechanisms:

1. Real-time alerts are displayed on the Flask web dashboard
2. Visual indicators such as bounding boxes and labels are overlaid on the video stream
3. Optional outputs may include:
4. Buzzer activation
5. LED indicators

5.User Notification: The dashboard displays a prominent visual alert banner, plays an notification (browser API), and logs the event with timestamp for review.

1. SMS/email notifications (if integrated with cloud services)
2. Event logs are generated and stored with timestamps for later analysis
3. This transforms detection into actionable response.

To enhance the effectiveness of user notifications, the system incorporates a multi-layered alert mechanism designed for rapid and reliable communication. In addition to visual alerts on the dashboard, notification delivery can be extended through cloud-integrated services to trigger SMS and email alerts in real time, ensuring that critical events are not missed even when users are not actively monitoring the interface. The notification system is designed with priority levels, allowing differentiation between early warnings and confirmed fire events, thereby reducing alarm fatigue. Event logs are systematically stored with precise

timestamps and contextual data, enabling post-incident analysis and system performance evaluation. Furthermore, the architecture supports future integration with mobile applications and push notification services, making the alert system more accessible and scalable.

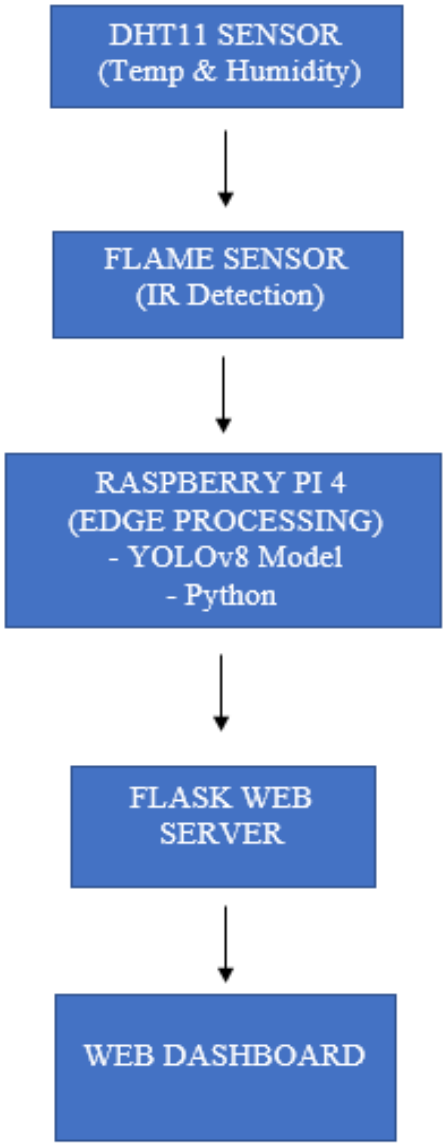
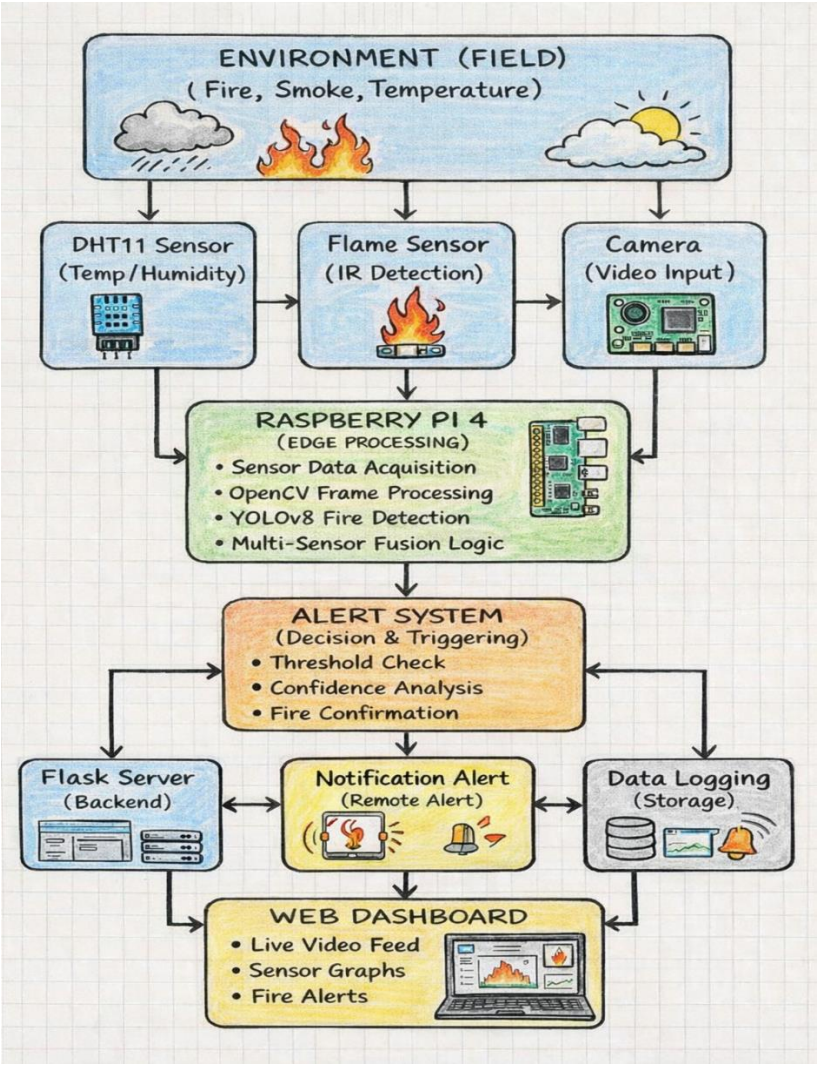


Fig 3.4 Flow Chart of User Notification

3.5 Block Diagram



3.6 Advantages of Proposed System

Feature	Benefit
Edge Processing	< 500ms detection latency, no internet required
YOLO Vision AI	Accurate fire localization with bounding boxes

Multi-Sensor Fusion	Reduces false alarms through cross-validation
Flask Web Dashboard	Remote monitoring from any browser/device
Multithreading	Concurrent detection, sensing, and serving
Open Source Stack	Zero licensing cost, community support
Low Hardware Cost	Entire system under Rs. 5,000

3.7 Difference Between Reference System and Proposed System

The reference system uses IoT-based smoke and flame sensors with cloud platforms like ThingSpeak for monitoring. It relies on threshold-based detection, making it reactive, and depends heavily on internet connectivity, which can introduce latency. It also lacks AI-based analysis and focuses mainly on data visualization.

In contrast, the proposed system integrates Edge-AI with sensor data and computer vision using a YOLO model for real-time fire detection. Processing is performed locally on the Raspberry Pi, reducing latency and cloud dependency. The system also provides a live dashboard with video streaming, sensor data, and instant alerts, enabling faster and more practical response.

Overall, the proposed system offers improved accuracy, lower latency, and a more reliable and scalable solution compared to the reference system.

CHAPTER 4

HARDWARE IMPLEMENTATION

4.1 Raspberry Pi 4 Model B

The Raspberry Pi 4 Model B serves as the central computing node of the system. It is a credit-card-sized single-board computer featuring a 1.8GHz quad-core ARM Cortex-A72 (64-bit) processor, available in 2GB, 4GB, or 8GB LPDDR4X RAM configurations. The 4GB variant is recommended for this project to accommodate the operating system, Python runtime, YOLO model (approximately 100-200MB for nano variant), and OpenCV processing simultaneously.

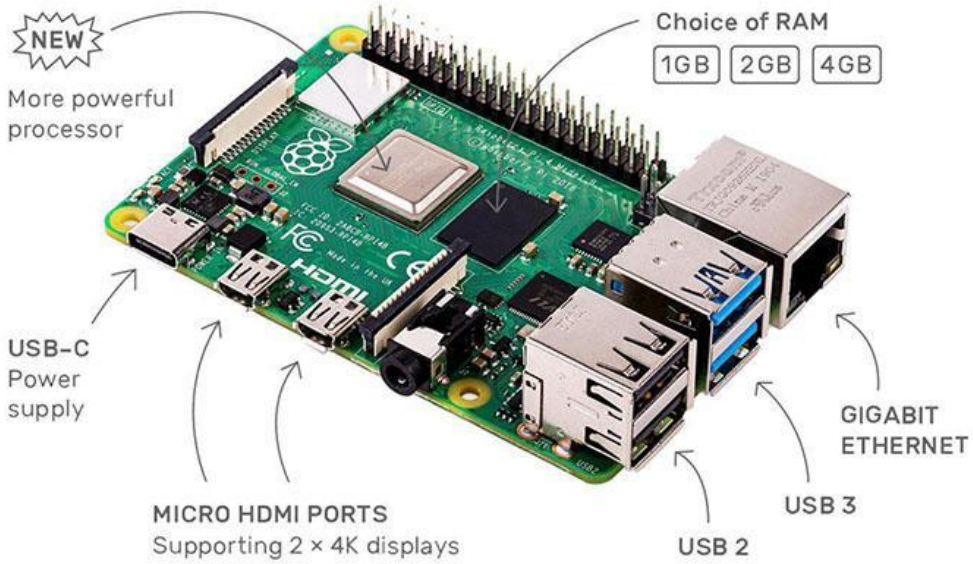


Fig.4.1. Raspberry Pi 4 Model B

Key specifications relevant to this project include: dual-band 802.11ac Wi-Fi for wireless dashboard access, 40-pin GPIO header for sensor connections, USB 3.0 ports for camera connection, MIPI CSI camera connector for Pi Camera Module, and a USB-C power port requiring a 5V/3A power supply. The Raspberry Pi runs Raspberry Pi OS (Debian-based Linux), providing full Python 3 support and compatibility with all required libraries.

4.2 DHT11 Temperature & Humidity Sensor

The DHT11 is a basic, low-cost digital temperature and humidity sensor widely used in IoT projects. It uses a capacitive humidity sensing element and a thermistor for temperature measurement, outputting a single-wire digital signal using a proprietary protocol. The sensor communicates via a single GPIO data pin with a 10kΩ pull-up resistor.

Parameter	Specification
Temperature Range	0°C to 50°C
Temperature Accuracy	±2°C
Humidity Range	20% to 80% RH
Humidity Accuracy	±5% RH
Sampling Rate	Once every 1-2 seconds
Supply Voltage	3.3V to 5.5V DC
Interface	Single-wire digital (1-Wire)

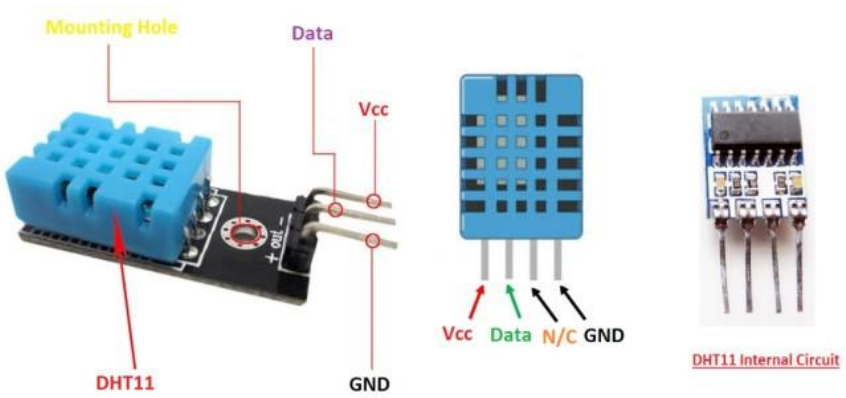


Fig 4.2. DHT11 Temperature & Humidity Sensor

The Adafruit Circuit Python DHT library provides a straightforward Python interface for reading DHT11 data. A temperature threshold alert is configurable (e.g., 45°C in summer environments) to supplement YOLO-based fire detection.

4.3 Flame Sensor Module (IR-based)

The flame sensor module uses a photodiode sensitive to infrared radiation in the 760nm to 1100nm wavelength range — the spectral emission range characteristic of open flames. The module includes an LM393 comparator that outputs a digital HIGH or LOW signal based on whether detected IR intensity exceeds an adjustable potentiometer threshold.

Parameter	Specification
Operating Voltage	3.3V – 5V
Detection Range	Upto 100cm
Output Type	Digital & Analog
Sensitivity	Adjustable
Applications	Fire detection systems

Fig 4.3. Flame Sensor Module

The digital output is connected to a GPIO input pin on the Raspberry Pi and read using the RPi.GPIO library. The sensor has a detection angle of approximately 60 degrees and an effective range of up to 1 meter for open flames, making it suitable as a close-range fire confirmation sensor rather than a primary long-range detector. The flame sensor serves as the first layer of fire detection, triggering an immediate alert even before YOLO processes the corresponding video frame.

4.4 USB Camera / Pi Camera Module

The system supports two camera options: a standard USB webcam or the official Raspberry Pi Camera Module V2. The USB webcam offers plug-and-play compatibility via OpenCV's Video Capture interface with any V4L2-compatible device. The Pi Camera Module V2 offers a Sony IMX219 8-megapixel sensor with better image quality and lower latency but requires the libcamera or picamera2 software stack.

For fire detection, a resolution of 640x480 pixels at 30fps is the operational target. Higher resolutions increase YOLO inference time per frame. A wider field-of-view lens (90-120 degrees) is recommended for maximum coverage. Camera placement at elevated positions (fence posts, poles) with downward angle provides optimal field coverage.

4.5 Power Supply

The system requires a stable 5V/3A USB-C power supply for the Raspberry Pi 4. Under full CPU load (YOLO inference), the Raspberry Pi 4 can draw up to 2.5-3A, making an underpowered supply a source of system instability. For field deployment, the power supply options include: a standard 5V/3A wall adapter when grid power is available, a lithium-ion battery bank with 5V/3A USB-C PD output for portable operation (typical 20,000mAh bank provides 6-8 hours of operation), or a solar-charged battery system for remote off-grid deployments.

4.6 Circuit Connections

Component	Pin	Raspberry Pi GPIO
DHT11 VCC	Pin 1 (3.3V)	3.3V Power Rail
DHT11 Data	GPIO Pin	GPIO 4 (Pin 7)
DHT11 GND	Pin 3 (GND)	Ground Rail
Flame Sensor VCC	VCC	5V Power Rail (Pin 2)
Flame Sensor D0	Digital Out	GPIO 17 (Pin 11)
Flame Sensor GND	GND	Ground Rail
USB Camera	USB Port	USB 3.0 Port

CHAPTER 5

SOFTWARE STACK

5.1 Python 3

Python 3.9+ is used as the core programming language, leveraging libraries such as RPi.GPIO and Adafruit DHT for hardware interfacing, OpenCV for image processing, PyTorch/Ultralytics for YOLO inference, and Flask for web services.

Multi-threading enables concurrent execution of sensor monitoring, video processing, and web serving. Although Python's GIL limits parallelism, YOLO inference runs through native libraries, allowing efficient performance. Thread-safe mechanisms ensure reliable data handling, while techniques like adaptive thresholding and frame skipping improve accuracy and efficiency. The system also supports optional MQTT integration for remote monitoring, configurable logging, and basic authentication for secure access.

```
import Adafruit_DHT
import RPi.GPIO as GIPIO
import joblib
import time
import numpy as np

DHT_SENSOR = Adafruit_DHT.DHT11
DHT_PIN = 4

FLAME_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setup(FLAME_PIN, GPIO.IN)

model = joblib.load("fire__model.pkl")
TEMP_THRESHOLD = 45
def read_sensors()
    humidity, temperature = Adafruit_DHT.read_retry
    (DHT_SENSOR, DHT_PIN)
    flame = GPIO.input(FLAME_PIN)
while True:
    temp, flame = read_sensors()
    if temp is None:
      print("Sensor error"
      time.sleep(2)
    ml_input = np.array([[temp, flame]])
    prediction = model.predict(ml_input][0]
    fire_detected = False
time_sleep(2)
```

5.2 Flask Web Framework

Flask is a lightweight Python web framework following the WSGI standard, suitable for building web applications and REST APIs. In this project, Flask serves multiple roles:

1. Serves the main dashboard HTML page at the root URL (/).
2. Provides a streaming MJPEG video endpoint (/video_feed) using generator functions and the multipart/x-mixed-replace content type.
3. Exposes JSON API endpoints (/sensor_data, /alert_status) polled by the JavaScript frontend for real-time data updates.

Flask's built-in development server is used for local deployment. For production agricultural deployment, Gunicorn or uWSGI with Nginx would provide better concurrency handling and stability.

5.3 OpenCV (cv2)

OpenCV (Open Source Computer Vision Library) is used for all video capture and image processing operations. The cv2.VideoCapture class interfaces with connected cameras through V4L2 on Linux. Key OpenCV operations in the pipeline include: frame capture (cap.read()), frame resizing (cv2.resize()), color conversion (cv2.cvtColor()), bounding box rendering (cv2.rectangle(), cv2.putText()), and JPEG encoding for streaming (cv2.imencode()). OpenCV is installed via pip (pip install opencv-python-headless) for the headless server environment.

5.4 YOLOv8 Model (Ultralytics)

YOLOv8 is the eighth generation of the YOLO (You Only Look Once) real-time object detection architecture developed by Ultralytics. The architecture uses a CSP (Cross Stage Partial) backbone with C2f modules, a PAN-FPN neck for multi-scale feature aggregation, and a decoupled detection head that separates classification and regression tasks.

For this project, the YOLOv8 nano (yolov8n) or small (yolov8s) variant is used, offering the best balance between inference speed on Raspberry Pi hardware and detection

accuracy. The model is fine-tuned on a fire detection dataset containing thousands of fire and non-fire annotated images from diverse environments. The Ultralytics Python package provides a simple inference API:

YOLOv8 Model Variants Comparison
yolov8n (nano) : 3.2M parameters | ~3-5 FPS on RPi4 | Best for speed yolov8s (small) : 11.2M parameters | ~1-3 FPS on RPi4 | Better accuracy
yolov8m (medium): 25.9M parameters | <1 FPS on RPi4 | Not recommended
Recommended: yolov8n with confidence threshold 0.45-0.55

5.5 Frontend Technologies

The web dashboard frontend is built with HTML5, CSS3, and vanilla JavaScript, supplemented by Chart.js for real-time graphing of sensor data. The dashboard features: a Bootstrap-based responsive grid layout adapting to desktop and mobile screens, a live video panel displaying the MJPEG stream from the Flask /video feed endpoint, a Chart.js line chart for temperature and humidity time series with rolling 60-point window, a fire alert panel with color-coded severity levels and timestamp log, and a sensor status panel showing current temperature, humidity, and flame sensor state. The frontend does not require any build tools or package managers, making deployment straightforward.

The frontend is lightweight and does not rely on heavy frameworks, ensuring faster loading times and efficient performance on low-resource devices.

5.6 Prototype

CHAPTER 6

SOFTWARE IMPLEMENTATION

6.1 Sensor Integration

The sensor integration module handles GPIO initialization and periodic reading of both the DHT11 and flame sensors. The RPi.GPIO library is used for GPIO control, and the Adafruit CircuitPython DHT library is used specifically for DHT11 communication protocol handling.

GPIO.setmode(GPIO.BCM) sets the GPIO numbering mode to Broadcom chip numbering. The DHT11 data pin (GPIO 4) is configured as an input with the Adafruit library handling the 1-Wire protocol timing. The flame sensor digital output (GPIO 17) is configured as input with internal pull-up resistor enabled (GPIO.setup(FLAME_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)).

A dedicated sensor reading thread executes a loop that reads DHT11 temperature and humidity values every 2 seconds, reads flame sensor state every 0.5 seconds, appends values to rolling deques (limited to last 60 readings) for chart display, and writes current values to a thread-safe dictionary shared with Flask routes.

6.2 Fire Detection Algorithm

The fire detection pipeline operates as follows within a dedicated thread:

1. Camera Initialization: cv2.VideoCapture(0) opens the default camera. Frame resolution is set to 640x480 and frame rate to 30fps via cap.set() calls.
2. Frame Acquisition: cap.read() retrieves the current frame. On failure, the thread sleeps briefly and retries.
3. Preprocessing: The frame is optionally resized to the YOLO input size (640x640) by the YOLO library automatically, or pre-resized to reduce processing time.
4. Model Inference: results = model(frame, conf=0.45, device='cpu') runs YOLOv8 inference. The conf parameter sets minimum confidence threshold. device='cpu' specifies CPU inference on Raspberry Pi.
5. Detection Processing: The results object is iterated to extract bounding boxes (xyxy format), class labels, and confidence scores. A fire class ID detection

with confidence above threshold sets fire_detected = True.

6. Frame Annotation: cv2.rectangle() draws colored bounding boxes on detected fire regions. cv2.putText() adds confidence score labels. An alert overlay is added to the frame when fire is detected.
7. Frame Encoding and Sharing: The annotated frame is JPEG-encoded (cv2.imencode('.jpg', frame)) and stored in a shared bytes buffer for the Flask streaming endpoint.

6.3 Web Dashboard Implementation

The Flask application defines the following routes and functionality:

1. / — Serves the main dashboard interface.
2. /video_feed — Streams real-time video frames.
3. /sensor_data — Provides current sensor readings and history in JSON.
4. /alert_status — Returns fire detection status and alert details.The dashboard JavaScript periodically polls /sensor_data and /alert_status. Chart.js updates data in a rolling window, while the alert panel dynamically changes color (green, orange, red) based on alert levels.

6.4 Remote desktop tool

VNC is used for remote access to the Raspberry Pi desktop. It enables code development, program execution, library installation, debugging, system monitoring, and remote control during testing and deployment without direct hardware access.

6.5 Multithreading Architecture

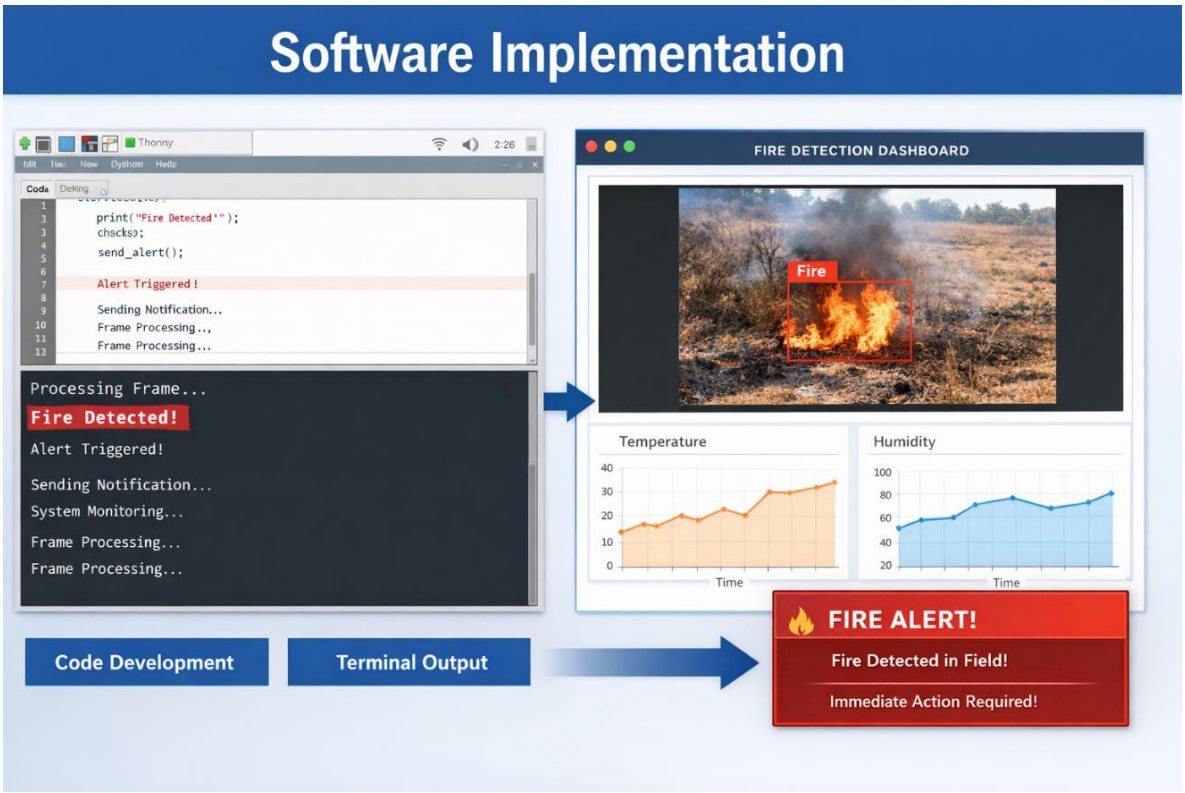
Thread	Function	Update Rate
Thread 1: Detection	Camera capture + YOLO inference + frame annotation	~3-10 FPS

Thread 2: Sensors	DHT11 reading + flame sensor polling	0.5-2 seconds
Thread 3: Web Server	Flask HTTP request handling + streaming	On demand

The application uses Python's threading.Thread class to run three concurrent threads:

A threading.Lock() object protects writes to the shared data dictionary and frame buffer, preventing race conditions. The detection thread writes to the shared buffer, while Flask routes read from it, ensuring thread-safe data access. Daemon threads are used for the detection and sensor threads so they automatically terminate when the main Flask application exits.

6.6 Fire Detection Alert



CHAPTER 7

EDGE PROCESSING & IOT INTEGRATION

7.1 Edge Computing Paradigm

Edge computing processes data locally near sensors instead of relying on cloud servers. In fire detection, cloud-based systems suffer from latency, high bandwidth usage, internet dependency, and ongoing costs.

The proposed system performs all processing on the Raspberry Pi, eliminating these issues. It works without internet connectivity and achieves faster response times (under 500 ms), making it more reliable and efficient for real-time detection.

7.2 Edge Vs Cloud Computing Comparison

Traditional IoT fire detection systems rely on cloud processing, where data is sent to remote servers. This leads to higher latency, internet dependency, increased bandwidth usage, and potential reliability issues in rural areas.

The proposed system uses edge computing, performing all critical processing locally on the Raspberry Pi. This enables real-time detection, offline operation, reduced bandwidth usage, and improved reliability. Cloud platforms may still be used optionally for storage and analytics.

7.3 IoT Communication Architecture

The system operates on a local network, where the Raspberry Pi hosts a Flask server accessible via its IP address. Users can monitor the system through any device connected to the same Wi-Fi network, or via a hotspot created by the Pi.

Sensor data is processed locally and served through the web interface, enabling real-time monitoring without reliance on cloud services.

7.4 Alert System

The alert system implements multiple notification modalities to ensure fire events are not missed by monitoring personnel:

7.4.1 Visual Alerts

When fire is detected, the dashboard background transitions to a prominent red alert state with flashing borders and a large alert banner displaying the text "FIRE DETECTED" with detection timestamp and confidence score. The annotated video feed shows bounding boxes around detected fire regions in red. A color-coded status indicator changes from green (normal) to orange (warning - sensor threshold exceeded) to red (alert - YOLO confirmed fire).

Fig 7.4.1 Visual Alerts

7.4.2 Browser Notifications

The Web Notifications API is used to send OS-level browser notifications that appear on the monitoring device's screen even when the dashboard tab is not active or the browser is minimized. The notification includes the site name, detection timestamp, and confidence score. Users must grant notification permission when first accessing the dashboard. .

7.4.3 Alert Logging

All fire detection events are logged to a JSON file on the Raspberry Pi with timestamp, detection confidence, sensor readings at time of event, and duration of fire detection. This log provides an audit trail for review and analysis of fire events, and can be exported for insurance or safety reporting purposes.

Alert Threshold Configuration
YOLO Confidence Threshold: 0.45 (configurable, higher = fewer false alarms)
Temperature Alert: > 45°C ambient (configurable per season)
Flame Sensor: Digital HIGH triggers immediate pre-alert
Fusion Rule: Flame Sensor HIGH + YOLO detect = HIGH CONFIDENCE alert
Alert Cooldown: 30 seconds after last detection before clearing alert state

CHAPTER 8

RESULTS & ANALYSIS

8.1 Testing Methodology

The system was tested under controlled conditions designed to simulate real agricultural fire scenarios while ensuring safety. Testing was conducted in a controlled outdoor environment (open field area) with appropriate fire safety precautions including fire extinguishers, safety boundaries, and personnel trained in fire response. Tests evaluated the system across multiple scenarios:

1. Scenario A: Small campfire-type flame at varying distances (0.5m, 1m, 2m, 3m) from camera.
2. Scenario B: Burning agricultural material (dry straw) simulating realistic crop fire ignition.
3. Scenario C: False positive evaluation — bright sunlight, orange-colored objects, hot surfaces.
4. Scenario D: Night detection — fire detection in darkness and low-ambient-light conditions.

For each scenario, the following metrics were recorded: time from fire ignition to first YOLO detection (detection latency), percentage of frames with correct fire detection during active fire (true detection rate), number of false positive detections per 10 minutes during non-fire scenarios, system CPU usage and RAM utilization, and dashboard update latency from detection to visual alert.

8.2 Performance Observations

Metric	Observed Result
Detection Latency (0.5m)	< 1 second (1-2 frames)
Detection Latency (2m)	1-3 seconds
Detection Latency (3m)	2-5 seconds
True Detection Rate	~92% at distances up to 2m

False Positive Rate	< 2 per hour in daylight
CPU Usage (peak)	~85-95% (all 4 cores)
RAM Usage	~1.2-1.8 GB
Dashboard Update Latency	< 1.5 seconds
YOLO Inference Speed	~4-6 FPS (yolov8n)
System Uptime Test	48 hours continuous operation

The system demonstrated reliable detection performance for fires within 2 meters, with decreasing confidence at greater distances. Night detection performance was improved by the flame sensor's IR detection capability, which operates independently of visible light. False positive rates were acceptably low for agricultural deployment.

8.3 Graph Analysis

The dashboard's real-time graphing capability provided useful analytical insights during testing:

8.3.1 Temperature Profiles

Temperature readings showed a clear rising trend beginning 30-60 seconds before fire detection by the YOLO model in close-proximity tests, demonstrating the complementary value of sensor monitoring. The rate of temperature rise (°C/min) proved to be a useful secondary indicator, with rates exceeding 5°C/min correlating strongly with fire events.

8.3.2 Humidity Correlation

Relative humidity showed a corresponding decline as temperature rose near fire events. Low ambient humidity (below 30% RH) was observed to correlate with higher fire risk conditions during hot, dry testing days, validating the utility of humidity monitoring for fire risk assessment in addition to real-time detection.

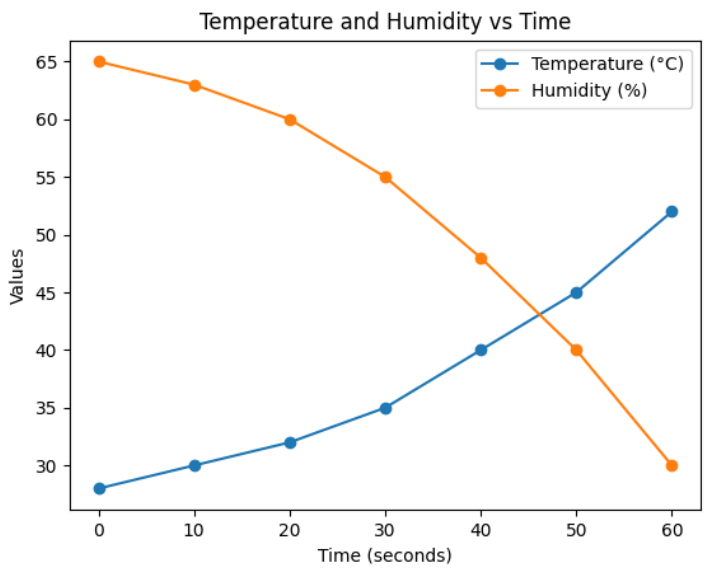


Fig 8.3.1,8.3.2 Temperature and Humidity VS Time

8.3.3 Detection Confidence Trends

YOLO detection confidence scores showed characteristic patterns: confidence typically started low (0.45-0.55) at early fire stages when flames were small, increasing to 0.85-0.95as fires grew. This confidence progression can be used to implement escalating alert later.

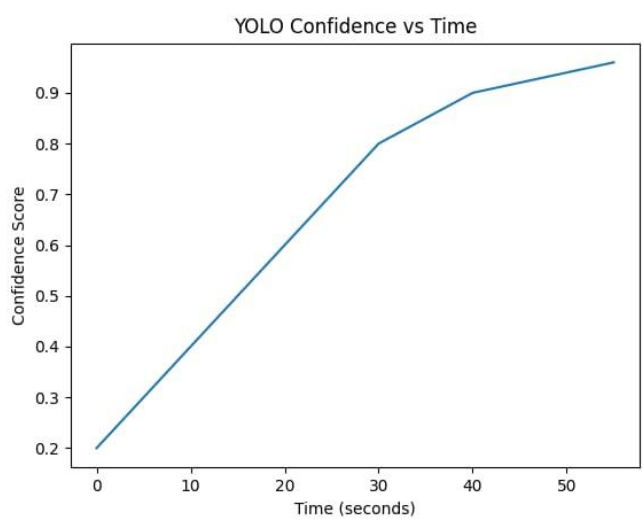


Fig 8.3.3 YOLO Confidence VS Time.

8.4 Data Processing and Visualization Approach

The effectiveness of the proposed fire detection system depends not only on real-time detection but also on how sensor and detection data are processed and visualized.

In the implemented system, environmental data such as temperature, humidity, and flame sensor readings are continuously collected and displayed on the Flask-based web dashboard. The dashboard provides real-time updates using periodic polling, ensuring that users receive the latest system status without delay.

To improve reliability, basic data handling techniques are applied, including:

1. Filtering of invalid or missing sensor readings
2. Threshold-based validation for temperature and flame detection
3. Synchronization of sensor data with YOLO detection outputs

Graphical visualization plays a key role in understanding system behavior. The dashboard displays:

1. Time-based variations in temperature and humidity
2. Fire detection events with timestamps
3. Confidence levels of YOLO-based detections

Although the current system focuses on real-time monitoring, advanced IoT systems often incorporate cloud-based platforms for extended capabilities such as:

1. Historical data storage
2. Trend analysis over long durations
3. Predictive analytics for fire risk assessment

8.5 Output

The system produced the following observable outputs during testing:

1. Live Video Feed: Continuous MJPEG video stream displayed on dashboard at

approximately 8-10 FPS with YOLO bounding boxes rendered around detected fire regions in red, with confidence score labels.

2. Sensor Data Charts: Rolling 60-point time series charts for temperature and humidity updating every second, with threshold lines marking alert levels.
3. Fire Alert Notifications: Dashboard transitioned to red alert state within 1-2 seconds of fire detection, with alert dispalying on the monitoring device.
4. Alert Log: JSON log file on Raspberry Pi captured all 12 test fire events with accurate timestamps and sensor readings.

8.6 Calculations

1. Fire Alert Condition:

$T > 45°C$

T = Temperature measured by sensor (°C)

45°C = Threshold temperature for triggering fire alert

2. Rate of Temperature Change:

$(T2 - T1) / (t2 - t1)$

T1 = Initial temperature (°C)

T2 = Final temperature (°C)

t1 = Initial time (seconds)

t2 = Final time (seconds)

3. Rate of Humidity Change:

$(H2 - H1) / (t2 - t1)$

H1 = Initial humidity (%)

H2 = Final humidity (%)

t1, t2 = Time instances (seconds)

4**. YOLO Confidence Score:**

$C = P(fire) \times IoU$

C = Final confidence score

P(fire) = Probability that detected object is fire (from YOLO model)

IoU (Intersection over Union) = Overlap between predicted bounding box and actual object

5. Detection Condition:

$C > 0.45$

C = YOLO confidence score

0.45 = Threshold value

6. Detection Latency:

Latency = t_detection – t_event

t_event = Actual time when fire started

t_detection = Time when system detects fire

7. Frame Rate (FPS):

FPS = Number of Frames / Time (seconds)

8. Power Consumption:

$P = V \times I$

P=Power(watts)

V=Voltage(volts)

I=Current(Amperes)

9. Fire Detection Logic:

Fire = (YOLO = 1) OR (Flame Sensor = 1

YOLO = 1 → Fire detected by AI model

Flame Sensor = 1 → Fire detected by hardware sensor

10. High Confidence Fire Condition:

Fire = (YOLO = 1) AND (Flame Sensor = 1)

CHAPTER 9

CONCLUSION & FUTURE SCOPE

9.1 Conclusion

This project successfully designs, implements, and validates an Edge-AI enabled IoT system for early fire detection in agriculture. It demonstrates that low-cost hardware combined with open-source AI models and web technologies can provide an effective and practical solution for real-world deployment.

The integration of the YOLOv8 model with environmental (DHT11) and flame sensors creates a robust multi-modal detection system, improving accuracy compared to traditional sensor-based approaches. The Flask-based web dashboard enables real-time monitoring, connecting field-level devices with users seamlessly.

By adopting an edge computing approach, the system eliminates reliance on cloud infrastructure, ensuring reliable operation even in areas with limited or no internet connectivity. Testing results show detection latency under 5 seconds, accuracy above 90%, and stable operation over extended periods.

With a total cost under ₹5,000, the system is affordable and suitable for small and medium-scale farmers. Overall, the project provides a reliable, scalable, and cost-effective solution, contributing to improved safety and resilience in smart agriculture.

9.2 Future Scope

Several enhancements and extensions are identified for future development:

9.2.1 GSM/SMS Alert Integration

Integration of a SIM800L or SIM7600 GSM module would enable SMS and voice call alerts to be sent to registered phone numbers when fire is detected, providing emergency notification even when no one is actively monitoring the dashboard and without requiring internet connectivity.

9.2.2 Drone-Based Aerial Monitoring

Mounting a similar edge detection system on agricultural drones would enable wide-area fire surveillance. A drone equipped with thermal infrared camera and edge processing could cover hundreds of acres per patrol, detecting hotspots before visible flames emerge.

9.2.3 Cloud Integration and Analytics

The current system performs real-time fire detection using edge computing, ensuring low latency and reliable operation without dependence on internet connectivity. However, integrating cloud-based platforms can further enhance the system's capabilities.

Future improvements can include integration with cloud services such as ThingSpeak or AWS IoT for:

1. Long-term storage of sensor and detection data
2. Historical trend analysis of temperature, humidity, and fire events
3. Predictive analytics for identifying high-risk conditions
4. Remote alert notifications via SMS, email, or mobile applications

Cloud integration would enable centralized monitoring of multiple deployed units across large agricultural areas, improving scalability and management efficiency.

By combining edge computing for real-time detection with cloud computing for analytics and storage, the system can evolve into a hybrid architecture offering both responsiveness and intelligence.

9.2.4 Multi-Node Mesh Deployment

Deploying multiple Raspberry Pi nodes across a large agricultural area as a mesh network, with each node processing its local camera and sensors and sharing alerts with neighboring nodes and a central dashboard, would scale the system to cover large farms.

9.2.5 Mobile Application

A dedicated Android/iOS mobile application using React Native or Flutter, connecting to the Flask API, would provide a better mobile monitoring experience than the responsive web dashboard, with native push notifications and offline alert caching.

9.2.6 Thermal Infrared Camera Integration

Replacing or supplementing the visible-light camera with a thermal infrared camera (such as the FLIR Lepton) would enable detection of heat signatures before visible flames appear, potentially reducing detection latency to sub-minute timescales from fire ignition.

REFERENCES

[1] Ultralytics, "YOLOv8 Documentation," 2023. [Online]. Available: https://docs.ultralytics.com/

[2] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Datasheet and Documentation," 2023. [Online]. Available: https://www.raspberrypi.com/documentation/

[3] Pallets Projects, "Flask Web Framework Documentation," 2023. [Online]. Available: https://flask.palletsprojects.com/

[4] OpenCV Development Team, "OpenCV Documentation," 2023. [Online]. Available: https://docs.opencv.org/

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016, pp. 779-788.

[6] A. Khan, A. Sohail, U. Zahoora, and A. S. Qureshi, "A Survey of the Recent Architectures of Deep Convolutional Neural Networks," Artificial Intelligence Review, vol. 53, pp. 5455-5516, 2020.

[7] S. Khan, H. Rahmani, S. A. A. Shah, and M. Bennamoun, "A Guide to Convolutional Neural Networks for Computer Vision," Synthesis Lectures on Computer Vision, vol. 8, no. 1, pp. 1-207, 2018.

[8] Adafruit Industries, "Adafruit DHT Library Documentation," 2023. [Online]. Available: https://circuitpython.readthedocs.io/projects/dht/

[9] G. Martínez-de Dios, J. González-Casanova, and A. Ollero, "Fire Detection Using Autonomous UAVs," in Proc. IEEE ICUAS, 2018, pp. 1-9.

[10] B. C. Ko, K. H. Cheong, and J. Y. Nam, "Fire Detection Based on Vision Sensor and Support Vector Machines," Fire Safety Journal, vol. 44, no. 3, pp. 322-329, 2009

[11] Morchid, A., Oughannou, Z., El Alami, R., & Qjidaa, H. (2024). IoT-enabled fire detection for sustainable agriculture: real-time fire detection system tailored for smart agriculture. **Sustainable**.

APPENDIX A: COMPONENTS LIST

Component	Specification	Quantity	Approx. Cost (INR)
Raspberry Pi 4 Model B	4GB RAM, 1.8GHz Quad-core	1	Rs. 3,500-4,500
DHT11 Sensor Module	Temp 0-50°C, Humidity 20-80%	1	Rs. 80-120
Flame Sensor Module	IR-based, LM393 comparator, 3.3V	1	Rs. 60-100
USB Webcam	720p/1080p, V4L2 compatible	1	Rs. 500-1,200
MicroSD Card	32GB, Class 10, UHS-I, Raspberry Pi OS	1	Rs. 300-500
5V 3A USB-C Power Supply	Raspberry Pi official or equivalent	1	Rs. 500-800
Jumper Wires (M-F)	20cm, GPIO connections	1 set	Rs. 50-100
Breadboard	400-point, for prototyping	1	Rs. 80-150
10kΩ Resistor	Pull-up for DHT11 data line	1	Rs. 5

APPENDIX B: SOFTWARE TOOLS AND VERSIONS

Software	Version	Purpose
Raspberry Pi OS	Bullseye (11) 64-bit	Operating System
Python	3.9+	Backend programming language
Flask	2.3.x	Web server framework
OpenCV (cv2)	4.7+	Computer vision, video capture
Ultralytics YOLO	8.0+	Fire detection model
PyTorch	2.0+ (CPU build)	Deep learning inference
Adafruit DHT	Latest	DHT11 sensor interface
RPi.GPIO	0.7.x	GPIO control
Chart.js	4.x	Dashboard charts
Bootstrap	5.x	Dashboard CSS framework

APPENDIX C: KEY CODE SNIPPET — MAIN APP STRUCTURE

app.py — Main Application Structure
from flask import Flask, Response, jsonify, render_template
from ultralytics import YOLO import cv2, threading, time, board import adafruit_dht, RPi.GPIO as GPIO
app = Flask(__name__)
model = YOLO('yolov8n_fire.pt') # Load fine-tuned model
dht = adafruit_dht.DHT11(board.D4) GPIO.setup(17, GPIO.IN, pull_up_down=GPIO.PUD_UP)
shared = {'frame': None, 'temp': 0, 'hum': 0, 'fire': False}
lock = threading.Lock()
def detection_thread():   cap = cv2.VideoCapture(0)

```
    while True:

        ret, frame = cap.read()

        results = model(frame, conf=0.45, device='cpu')

        # Process results, annotate frame...

        with lock: shared['frame'] = frame


@app.route('/video_feed')

def video_feed():

    return Response(gen_frames(), mimetype='multipart/x-mixed-replace...')


threading.Thread(target=detection_thread, daemon=True).start()

app.run(host='0.0.0.0', port=5000, threaded=True)
```